FIRST ASSIGNMENT :

1. Basic Arithmetic and Type identification : a. Create three variables: One Integers, One Float and One complex numbers print each variable and use the type() function to confirm their data types.

In [10]:
```
a = 32
a
print(type(a))

b = 3.00
b
print(type(b))

c = 3 - 4j
c
print(type(c))
```

```
<class 'int'>
<class 'float'>
<class 'complex'>
```

2. Arithmetic with mixed types : a. Create one int variable(a) and one float variable(b). print the sum, difference, product and quotient of a and b print the type() of each result(notice how the types may change)

In [11]:
```
a = 12
b = 16.12
```

In [12]:
```
# Sum of the variable:
sum = a + b
print(f"The sum of the {a} and {b} is {sum}")
print(type(sum))

# Difference of the variables :
diff = a - b
print(f"The difference between two variables {a} and {b} is {diff}")
print(type(diff))

# Product of the variables :
pro = a * b
print(f"The product of two variables {a} and {b} is {pro}")
print(type(pro))

# Quotient variables :
quo = a / b
print(f"The quotient of two variables {a} and {b} is {quo}")
print(type(quo))
```

```
The sum of the 12 and 16.12 is 28.12
<class 'float'>
The difference between two variables 12 and 16.12 is -4.120000000000001
<class 'float'>
The product of two variables 12 and 16.12 is 193.44
<class 'float'>
The quotient of two variables 12 and 16.12 is 0.7444168734491314
<class 'float'>
```

3. Comparison operator : a. Let x = 10 and y = 7 print the result of x>y, x<y, x==y, and x!=y After observing the output, explain why each result is True or False in comments

In [13]:
```
x = 10
y = 7
```

In [14]:
```
print(x > y) # The output gives True because in the variables x and y, x is greater
print(x < y) # The output gives False because in the variables x and y, x is greate
print(x == y) # The output gives False because the variable x is not equals to y
print(x != y) # The output gives True because the variable x is not equals to y("!"
```

```
True
False
False
True
```

4. Boolean Variables : a. Define a variable status = True. Print the value of status and confirm it is a boolean using type(status). Reassign status to False and confirm its type again.

In [15]:
```
status = True
print(type(status)) # Yes, it is a boolean data types
```

```
<class 'bool'>
```

In [16]:
```
status = False
print(type(status)) # Yes, This is also a boolean data types
```

```
<class 'bool'>
```

5. String Creation and Length: a.Create a string variable, for example text = "Hello World". Use len(text) to print its length. Use type(text) to verify it is a string.

In [17]:
```
name = "Bishal Tamang" # The characters are only 12 but it also counts the space so
print(len(name))
print(type(name))
```

```
13
<class 'str'>
```

6. String Indexing : a.With the string s = "Python", print each character. Then print just the first and last characters using negative indexing.

In [18]:
```
s = "Python"
print(len(s))
print(s[0])
print(s[1])
print(s[2])
print(s[3])
print(s[4])
print(s[5])

# For printing first elements:
print(s[0])

# For printing last elements:
print(s[-1])
```

```
6
P
y
t
h
o
n
P
n
```

7. String Slicing : Let lang = "Programming". Print the substring from index 0 to index 4. Print the substring from index 3 to the end. Print the substring that omits the first 2 and last 2 characters.

In [19]:
```
lang = "Programming"
print(lang[0:5]) # To access till 4th index we have to give the range of 5 because
print(lang[3:])
print(lang[2:-2])
```

```
Progr
gramming
ogrammi
```

8. Exploring len() in slices : a. Continue using lang = "Programming". Slice lang to get "Program" and store it in a variable part1. Print len(part1) and comment how it differs from len(lang).

In [20]:
```
lang = "Programming"
print(len(lang))
part1 = lang[0:7]
print(part1)
```

```
11
Program
```

In [21]:
```
print(len(part1))
```

```
7
```

In [22]:
```
# len(part1) is differ from len(lang) because len(part1) is sliced string/variable
# len(lang) is original string/variable which contains 11 characters
```

8. String Methods – Case Conversion a. Let phrase = "Hello, Python!". Print phrase.upper() and phrase.lower(). Print the original phrase to show it remains unchanged (strings are immutable).

In [23]:
```
phrase = "Hello, Python!"
print(phrase.upper()) # It makes all string's elements capital temporarily whereas
print(phrase.lower()) # It makes all string's elements smaller/ lower temporarily
print(phrase) # Strings are same as before, these upper and lower function changes
                                                                    #because strings are im
```

```
HELLO, PYTHON!
hello, python!
Hello, Python!
```

9. Combining Strings : a. Create two strings, str1 = "Data" and str2 = "Science". Combine them into a single string with a space in between and print it. Print the length of the combined string.

In [24]:
```
str1 = "Data "
str2 = "Science"
combining = str1 + str2 # Combining two variables doing sum
print(combining)
print(len(combining))
```

```
Data Science
12
```

10. In-Place vs. Reassignment with String Methods a. Let word = "example". Call word.upper() but do not store it, then print word. Now set word = word.upper(), then print word. Comment on why the second print is different from the first.

In [25]:
```
word = "example"
word.upper()
print(word)
print(word.upper())
```

```
example
EXAMPLE
```

In [26]:
```
# The second print is different from the first because:
    # In first case, .upper() create only new string. Since, we haven't store that,
    # And, In second case, after we printed out the variable and after that only we
    #          and modified the word "example" to "EXAMPLE"
```

11. Arithmetic Operator Precedence a. Define a = 5, b = 3, c = 2. Print the result of the expression a + b * c. Print the result of (a + b) * c. In comments, explain how operator

precedence affects the outcome.

In [27]:
```
a = 5
b = 3
c = 2
```

In [28]:
```
result = a + b * c
result1 = (a + b) * c
print(result)
print(result1)
```

```
11
16
```

In [29]:
```
# In the first expression, * gets higher priority according to mathematics so, b *
# And, in the second expression, "()" parenthesis gets the higher priority accordin
```

12. String Index Out of Range : a. Let text = "Hello". Attempt to access an index that doesn't exist, like text[10]. Observe the error message (IndexError) and explain why it happens in comments.

In [30]:
```
text = "Hello"
print(len(text))
print(text[10])
```

```
5
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[30], line 3
      1 text = "Hello"
      2 print(len(text))
----> 3 print(text[10])

IndexError: string index out of range
```

In [ ]:
```
# It happens because the given string's range is upto 5 only and we tried to access
```

13. Equality vs. Identity Check (Conceptual Explanation) : a. Create two variables with the same string value, s1 = "test" and s2 = "test". Use the == operator to compare them, then use the is operator. Explain in comments what each comparison checks.

In [ ]:
```
s1 = "test"
s2 = "test"
print(s1 == s2)
print(s1 is s2)
```

```
True
True
```

In [ ]:
```
# Both the variable gives True because
    # "==" operator checks whether the value of s1 and s2 is same or not and those
```

```
    # On the other side "is" checks whether the string consumes the same amount of
    #  But if we give a small space in one text, it will give false in both the ope
```

13. Creating and Checking a Complex Number : a. Define z = 3 + 4j. Print the real part (z.real) and the imaginary part (z.imag). Confirm that its type is complex using type(z).

In [ ]:
```
z = 3 + 4j # Complex number
print(z.real) # Real part
print(z.conjugate()) # Imaginary part
```

```
3.0
(3-4j)
```

In [ ]:
```
# Confirming the types :
print(type(z))
```

```
<class 'complex'>
```

14. Comparisons with Floats : Let f1 = 0.1 + 0.2 and f2 = 0.3. Print f1 == f2. Print the actual values of f1 and f2 to explain any difference in the comparison outcome (floating-point precision).

In [ ]:
```
f1 = 0.1 + 0.2
f2 = 0.3
print(f1 == f2)
print(f1)
print(f2)
```

```
False
0.30000000000000004
0.3
```

In [ ]:
```
# The output gives False, by adding 0.1 and 0.2 which doesn't gives the exact value
```

In [ ]:
```

```